



Budapesti Műszaki és Gazdaságtudományi Egyetem
Méréstechnika és Információs Rendszerek Tanszék

Teljesítményértékelő rendszer fejlesztése AI-integrációval

Önálló laboratórium zárójegyzőkönyv
2025/26. II. félév

Jász Barnabás

III. évf, mérnökinformatikus szakos hallgató
BSc Szoftverfejlesztés specializáció

Konzulensek:

Mészáros Tamás, docens
(Mesterséges Intelligencia és Rendszertervezés Tanszék)

Karz Gergely Jakab, Managing Director
(Artillence Kft.)

Feladatkiírás

A vállalati környezetben alkalmazott teljesítményértékelési módszerek – különösen a csapatalapú, kölcsönös értékelésen alapuló megközelítések – jelentős mennyiségű számszerű és szöveges adatot eredményeznek. Ezen visszajelzések feldolgozása, rendszerezése és értelmezése a HR-területen dolgozó szakemberek számára időigényes és összetett feladat, miközben a vezetők részéről egyre nagyobb az igény az átlátható, megalapozott és összehasonlítható teljesítményértékelések iránt.

A mesterséges intelligencia alapú megoldások képesek ezen visszajelzések elemzésére és összegzésére, ugyanakkor biztosítani kell, hogy az automatizált eszközök ne hozzanak önálló döntéseket az érintett személyekről, valamint az elemzések eredményei átláthatóak és visszakövethetőek maradjanak.

A feladat célja egy olyan mesterséges intelligenciával támogatott teljesítményértékelési rendszer megtervezése és megvalósítása, amely a teljesítményértékelési ciklusok kezelését és elemzését segíti vállalati környezetben. A rendszer a beérkező strukturált és strukturálatlan visszajelzések feldolgozásán keresztül támogatja a teljesítményértékelések különféle szempontok szerinti értelmezését, miközben biztosítja az emberi döntéshozatal elsődlegességét és az MI-alapú elemzések magyarázhatóságát.

A hallgató feladatai:

- Natív felhasználói felület megtervezése és kialakítása
- Példa kiértékelési kritériumok, szempontok megfogalmazása
- Integráció megvalósítása a főbb Google-alapú szolgáltatásokkal
- A beérkező visszajelzések AI-alapú feldolgozásának megtervezése és implementálása
- Dinamikus, AI-támogatott kiértékelési szempontok kialakítása
- Az eredmények strukturált, átlátható és értelmezhető prezentációs modelljének megtervezése
- Dokumentáció

A téma szakdolgozatnak, TDK-nak, diplomadolgozatnak továbbfejleszthető. Esetleges SOTA megoldás elérése esetén leaderboard nevezést támogatjuk.

Tartalom

Feladatkiírás	2
Bevezetés: a feladat értelmezése, a specifikáció pontosítása	4
A specifikáció pontosítása.....	4
Előzmények.....	5
A tervezés részletes leírása, a választott megoldások indoklása	6
1. Rendszerarchitektúra.....	6
2. Adatbázis Tervezés	7
2.1. Egyed-Kapcsolat (ER) Diagram:.....	7
3. Biztonság és Jogosultságkezelés	8
4. Fő Üzleti Folyamatok	9
5. Felhasználói Felület	11
6. Hibakezelés és Naplózás.....	13
A megtervezett műszaki alkotás értékelése, továbbfejlesztési lehetőségek	14
Továbbfejlesztési lehetőségek.....	14
Irodalomjegyzék.....	15

Bevezetés: a feladat értelmezése, a specifikáció pontosítása

A feladat egy olyan komplex, önállóan is működőképes webes alkalmazás tervezése és implementálása volt, amely a hagyományos HR adatbázis-kezelést ötvözi a legkorszerűbb (State of the Art) mesterséges intelligencia megoldásokkal. A keretrendszer két, szorosan összefüggő dologra épül:

1. **Adminisztrációs és adatgyűjtő keretrendszer:** A rendszernek képesnek kell lennie a dolgozók adatainak (név, e-mail cím, csoport, szerepkör) strukturált tárolására. Lehetőséget kell biztosítani a HR munkatársak számára kérdőívsablonok és egyedi kérdések összeállítására, valamint komplex értékelési kampányok menedzselésére. A kijelölt munkatársak magán a rendszeren belül tudják kitölteni az űrlapokat. A folyamatot automatizált e-mail értesítések támogatják, amelyek azonnal kimennek az érintetteknek, amint kiválasztják őket értékelőnek.
2. **LLM-alapú intelligens kiértékelés és tanácsadás:** A begyűjtött adatok (pontszámok és szabadszöveges válaszok) automatikus feldolgozása nyelvi modellek segítségével történik. Az AI ágensek feladata nem csupán a beérkezett válaszok szintetizálása, hanem az adatok alapján személyre szabott fejlődési lehetőségek megfogalmazása, továbbá a dolgozó aktuális teljesítményének összehasonlítása az előző kampányok eredményeivel, ezzel biztosítva a folyamatos nyomon követést.

A specifikáció pontosítása

A prototípus fejlesztése során az alábbi kibővített funkcionális és technikai követelményeknek kell megfelelni:

Funkcionális követelmények:

- **Jogosultság- és adatkezelés:** A rendszernek teljes körű Szerepkör (Role) és Jogosultság (Permission) rendszert kell implementálni CRUD elvek szerint. A rendszer nyilvántartja az alkalmazottakat, szerepköreiket és csoportjaikat.
- **Kérdőív- és kampánykezelés:** A felhasználóknak (HR) felülete van űrlapsablonok és kérdések létrehozására. A kampányok során dinamikus értékelési mátrixok épülnek fel (pl. beosztott értékeli a vezetőt, kolléga a kollégát). A kampányok paraméterei (pl. kiküldési arányok) futásidőben, az aktív szakasz alatt is módosíthatók.
- **Automatizált kommunikáció:** A munkafolyamat szerves részeként a rendszer automatikus e-mailt generál és küld azoknak a dolgozóknak, akiket egy kampányban értékelésre jelöltek ki.
- **Rendszeren belüli kitöltés:** A keretrendszer saját frontendet biztosít a felhasználóknak, ahol a kapott felkérések alapján, dedikált kérdőíveken keresztül végezhetik el a pontozást és a szöveges értékelést.
- **AI-vezérelt analitika és historikus összehasonlítás:** A kampány lezárultával az LLM kiértékeli a válaszokat. A nyers adatokból statisztikákat (pl. Stack Overflow felmérésekhez hasonló interaktív grafikonok) készít. Az LLM ezen felül detektálja a korábbi értékelésekhez viszonyított változásokat, trendeket, és konkrét fejlődési, tanulási javaslatokat fogalmaz meg az értékelt munkatárs számára.

Technológiai és architekturális követelmények:

- **Backend és Adatbázis:** A hordozhatóság érdekében a perzisztenciát relációs SQLite adatbázis biztosítja.
- **Frontend:** A felhasználói felület és az adatvizualizáció a Python alapú Streamlit keretrendszerrel készül.
- **Szoftverfejlesztési szabványok:** A függőségek szigorú verziókövetése (requirements.txt), valamint a platformfüggetlen futtathatóság biztosítása konténerizációval (Dockerfile) alapkövetelmény a prototípus átadhatósága érdekében.

Előzmények

A fejlesztési szakasz megkezdését megelőzően a projekt alapjainak lefektetése az alábbi elemző és tervező tevékenységekből állt:

- **Frontend keretrendszer alkalmasságának vizsgálata:** A Streamlit technológia alapos kiértékelése annak megállapítására, hogy funkcionálisan és skálázhatóság szempontjából mennyire felel meg a projekt által megkövetelt interaktív, adatvezérelt HR prototípus gyors és hatékony implementálására.
- **Értékelési mátrixok és adatszerkezetek elemzése:** A konzulens által kiadott, a szervezeti hozzárendelési mátrixot leíró Excel-dokumentumok áttanulmányozása. Az itt azonosított összefüggések alapján a rendszer adatmodelljére és a komplex (szerepkör-alapú) értékelési logikára vonatkozó kulcsfontosságú tervezési döntések meghozatala.
- **Kérdőíves folyamatok és sablonok felmérése:** A korábbi gyakorlatban használt Google Forms kérdőívek és az ezekhez kapcsolódó dokumentációk elemzése. A meglévő struktúrák alapján a keretrendszer saját, belső űrlapkészítő és kitöltő moduljának technikai és funkcionális megtervezése.
- **Szoftverarchitektúra és környezet megtervezése:** A platformfüggetlen működést biztosító konténerizációs (Docker) stratégia, valamint a transzparens függőségkezelési (requirements.txt) elvek lefektetése a modern, agilis szoftverfejlesztési irányelveknek megfelelően.
- **LLM modellek és keretrendszerek felmérése:** A piacon elérhető, költséghatékony (OpenRouter API-n keresztül elérhető ingyenes) nyelvi modellek és a hozzájuk kapcsolódó integrációs könyvtárak (pl. LangChain, Huggingface Transformers) képességeinek, valamint a projekt kiértékelési moduljába való beilleszthetőségüknek az elemzése.

A tervezés részletes leírása, a választott megoldások indoklása

1. Rendszerarchitektúra

1.1 Technológiai stack

A projekt a következő alaptechnológiákat használja, melyek kiválasztását a gyors iteráció, az adatközpontúság és az egyszerű karbantarthatóság indokolta:

- **Frontend és UI Keretrendszer: Streamlit**
 - A Streamlit lehetővé teszi interaktív, modern webes felhasználói felületek gyors kialakítását közvetlenül Python nyelven. Mivel a projekt erősen adatközpontú (űrlapok, értékelések, dolgozói listák), a Streamlit beépített komponensei (st.dataframe, formok, expanderek, párbeszédpanelek) felgyorsítják a fejlesztést anélkül, hogy különálló frontend (pl. React/Vue) stacket kellene fenntartani.
- **Backend és Üzleti Logika: Python (3.x)**
 - A Python olvashatósága, valamint a gazdag ökoszisztéma ideális választássá teszi. Kiemelten fontos szempont volt a LangChain és a mesterséges intelligencia (LLM) modellek (pl. OpenAI/OpenRouter) natív integrálhatósága a rendszerbe (pl. automatikus kérdés- és értékelésgenerálás céljából).
- **Perzisztens Adattárolás: SQLite**
 - Az SQLite egy fájlalapú, beágyazott relációs adatbázis-kezelő rendszer. Mivel nincsenek bonyolult szerverinfrastruktúra-követelményei, a rendszer könnyen hordozható és telepíthető. Megfelelő beállításokkal biztosítja az adatintegritást, és a jelenlegi skálázódási igényeket (belső vállalati használat) tökéletesen kiszolgálja.

1.1. Architekturális minta: Moduláris dizájn és rétegek

Bár a rendszer egyetlen alkalmazásként fut, a kódfelelősségek szét vannak választva. Ez a fájlstruktúrában és a modulok elnevezésében is megmutatkozik:

- **Megjelenítési és Nézeti Réteg:** Ebbe a rétegbe tartoznak a képernyőket és UI-elemeket generáló modulok. Ez a réteg felel a gombok, űrlapok, listák (pl. dolgozók vagy kampányok listája) Streamlit segítségével történő rendereléséért, és a felhasználói interakciók fogadásáért.
- **Üzleti Logika Réteg:** A *classes/* mappában található osztályok (pl. *User*, *FormTemplate*) fogják össze a rendszer magját képező logikát. Ezek az entitások reprezentálják a valós világ modelljeit, és biztosítják, hogy az adatbázisból kinyert sorok objektum-orientált módon, típusbiztosan legyenek feldolgozva.

1.2. Adatelérési Réteg: A *Database/* könyvtár moduljai (pl. *db_employee.py*, *db_forms.py*) rejtik el a nyers SQL lekérdezéseket.

- Egy központi *Context Manager* (*db_database_manager.py*) felel az adatbázis-kapcsolatok biztonságos megnyitásáért, a tranzakciók véglegesítéséért és hiba esetén a visszavonásért (rollback).
- A frontend réteg sosem hajt végre közvetlen SQL parancsot, mindig ezeken az interfészekon keresztül kommunikál.

1.3. Állapotkezelés

Mivel a Streamlit futtatási modellje "stateless" (egy interakció hatására a szkript felülről lefelé újra lefut), a munkamenetek (session) adatait és az állapotokat explicit módon kell kezelni a memóriában. A rendszer erre a beépített *st.session_state* objektumot használja:

- **Autentikáció és Bejelentkezett Felhasználó:** A bejelentkezés során azonosított felhasználó adatai (User objektum formájában) bekerülnek a *st.session_state.user* kulcs alá.
- **Jogosultságkezelés:** A *db_database_manager.py*-ban található *check_permission* függvény a munkamenetben lévő felhasználó szerepköre alapján dinamikusan ellenőrzi az adatbázisból, hogy a felhasználó jogosult-e egy adott műveletre.

(pl. create, update, delete). Ezen az alapon a UI elemek dinamikusan tiltásra kerülnek, ha nincs megfelelő jog.

- **Ideiglenes adatok és Nézetvezérlés:** A különböző űrlapok közötti navigáció és a komplex felületek állapota is itt tárolódik. Például a *st.session_state.current_form_id* határozza meg, melyik űrlap szerkesztőnézetét látja a felhasználó, míg a lapozás aktuális állását a *st.session_state.templates_page* tartja nyilván.
- **Gyorsítótárazás (Caching):** Az optimalizáció és az adatbázis tehermentesítése érdekében a lassan változó adatok (pl. összes sablon listája, AI generált értékek) *st.session_state*-ben és Streamlit *@st.cache_data* dekorátorokkal vannak eltárolva, amelyeket adatbázisváltozás esetén a rendszer manuálisan invalidál.

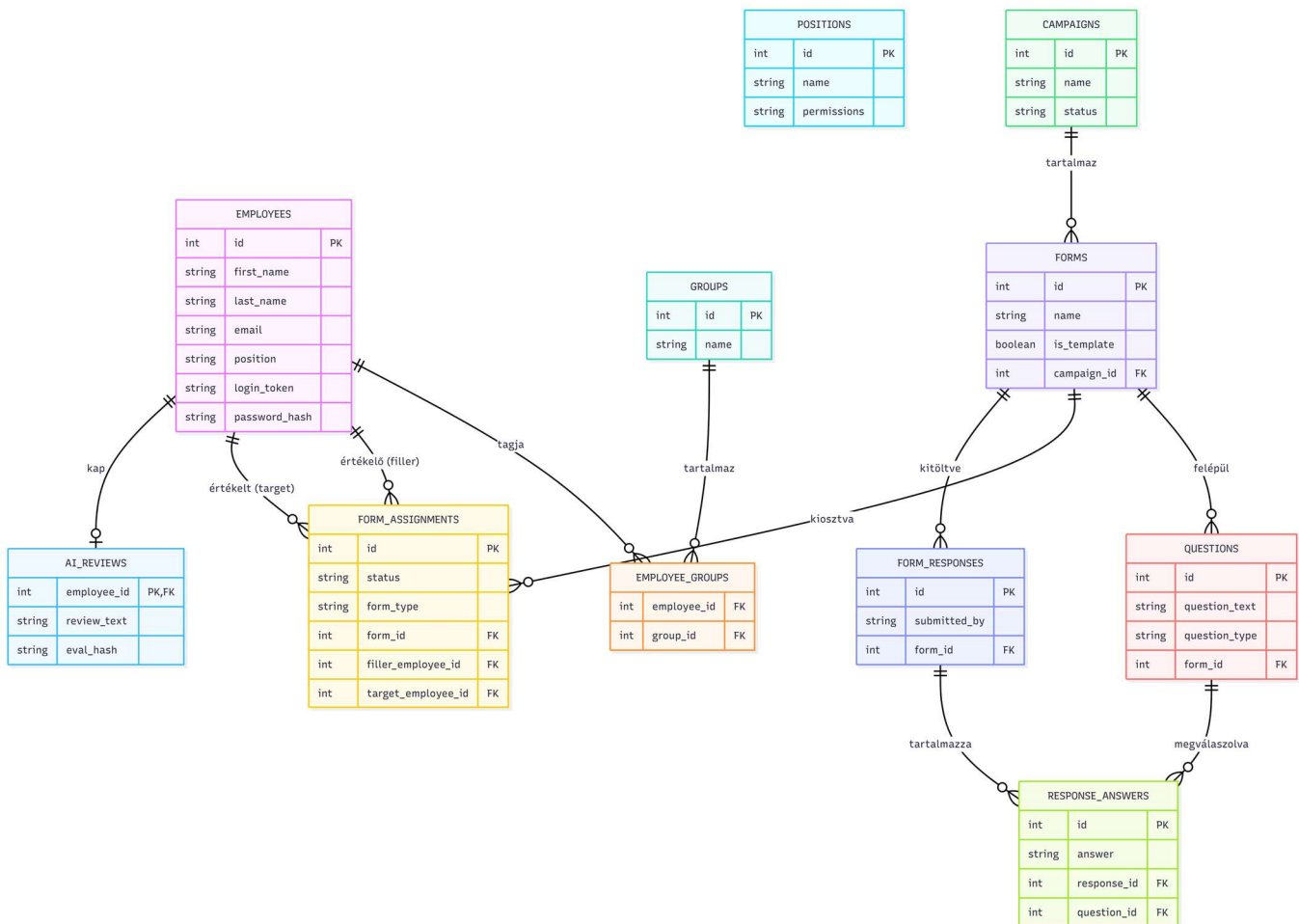
1.4. Mesterséges Intelligencia integrálása

MI integrálást OpenRouter és OpenAI könyvtárak segítségével lett bevezetve. OpenRouter szolgáltatón keresztül bizonyos token mennyiségig lehet használni modelleket ingyenesen. A tesztelések során a legalkalmasabb modell az openai/gpt-oss-20b:free modell volt. A rendszer képességeit és gyorsaságát még lehetne fejleszteni egy jobb modellel.

2. Adatbázis Tervezés

A rendszer adatelérési és perzisztencia rétege egy helyi SQLite adatbázisra (data.db) épül. Az architektúra központi eleme a relációs adatmodell, amely szigorú idegenkulcs-kényszerekkel biztosítja az adatintegritást. Az adatbázis kezelését a *db_database_manager.py* végzi, amely felelős a kapcsolatokért, a sémadefiníciókért és a migrációkért.

2.1. Egyed-Kapcsolat (ER) Diagram:



2.2. Főbb Táblák és Sémák Leírása

A sémát úgy lett tervezve, hogy három fő területet szolgáljon ki hatékonyan:

- **Felhasználó- és Jogosultságkezelés:** A dolgozók, a szervezeti egységek és a szerepkörök kezelése. A rendszer a pozíciókhoz rendelt dinamikus, vesszővel elválasztott

engedély-mátrix (permissions: *create, read, update, delete*) alapján biztosítja a szerepkör-alapú hozzáférés-vezérlést, CRUD elvek alapján.

- **Kampányok és Űrlapok:** A rendszer megkülönbözteti az általános űrlapsablonokat és az aktív értékelési kampányokhoz rendelt konkrét űrlapokat. Az űrlapok kérdésekből épülnek fel, és a *form_assignments* kapcsolótábla segítségével kerülnek kiosztásra a dolgozók között (nyilvántartva, hogy ki értékeli ki).
- **Válaszok és AI Értékelések:** A kitöltött értékelések fejléceit és a konkrét kérdésekre adott válaszokat relációs formában tárolja a rendszer. A mesterséges intelligencia által generált komplex szöveges összefoglalókat az *ai_reviews* tábla cache-eli. Ide egy egyedi azonosító (hash) is bekerül, amely a válaszokból képzett hash – így a rendszer képes felismerni, ha az adatok nem változtak, megelőzve a felesleges és költséges újra-generálást.
- **Adatbázis Migrációk és Séma Evolúció**
Az alkalmazás egy beépített migrációs mechanizmussal rendelkezik (*migrate_db()* függvény). Induláskor a rendszer dinamikusan ellenőrzi az SQLite adatbázis aktuális sémáját (*PRAGMA table_info* segítségével), és automatikusan végrehajtja a szükséges módosításokat (pl. új oszlopok hozzáadása, táblák átnevezése) adatvesztés nélkül. Ez biztosítja az egyszerű telepíthetőséget és a verziókövetést.
- **Biztonságos Kapcsolat- és Tranzakciókezelés**
Az adatbázis műveletek biztonságát és erőforrás-hatékonyságát a *db_connection()* context manager biztosítja
- **Adatintegritás:** A kapcsolat megnyitáskor automatikusan élesíti az idegenkulcs-kényszereket (*PRAGMA foreign_keys = ON*).
- **ACID Tranzakciók:** A műveletek blokkokban futnak le. Sikeres végrehajtás esetén automatikusan megtörténik a módosítások véglegesítése, míg bármilyen kivétel esetén a rendszer visszavonja azokat (rollback), elkerülve az inkonzisztens adatbázis-állapotokat.

3. Biztonság és Jogosultságkezelés

3.1. Autentikáció (Hitelesítés)

A felhasználók azonosítása és a munkamenetek (session) kezelése a *db_login.py* modul funkcióira támaszkodik. A rendszer kétféle hitelesítési mechanizmust kombinál:

- **E-mail és Jelszó alapú belépés:** A regisztrált munkatársak az e-mail címük és a jelszavuk megadásával tudnak belépni a rendszerbe. Az *authenticate_employee* függvény ellenőrzi az adatbázis *employees* táblájában a megadott e-mail címet, majd egyezés esetén összeveti a bekért jelszót az adatbázisban tárolt értékkel. Sikeres belépés esetén a rendszer visszaad egy felépített User objektumot, amely bekerül a Streamlit munkamenetébe (*st.session_state*).
- **Token-alapú azonosítás:** A rendszer támogatja a token-alapú, jelszó nélküli azonosítást is (egyedi értékelő linkek kiküldéséhez). A *generate_user_token* függvény egy biztonságos, egyedi UUID azonosítót generál. A *get_user_by_token* metódussal ezen token alapján a felhasználó közvetlenül és biztonságosan azonosítható.

3.2. Autorizáció

A hitelesítést követően a rendszernek el kell döntenie, hogy az adott felhasználó milyen műveleteket hajthat végre. Ezt egy Szerepkör-alapú hozzáférés-kezelési modell biztosítja, amelyet a *db_database_manager.py* vezérel.

3.3. Szerepkörök és Pozíciók

A jogosultságok nem egyéni felhasználókhöz, hanem a hierarchiában betöltött pozíciókhoz vannak rendelve. Az adatbázis migráció során (a *migrate_db* függvény által) a rendszer az alábbi alapértelmezett pozíciókat és CRUD (Create, Read, Update, Delete) engedélymátrixot hozza létre a *positions* táblában.

3.4. A jogosultság-ellenőrzés menete

A frontend megjelenítésének és a backend funkciók végrehajtásának őrzője a központosított `check_permission` függvény.

1. A függvény elsőként ellenőrzi, hogy van-e aktív munkamenet (`st.session_state.user`).
2. Lekéri az aktív felhasználó pozícióját (pl. "Manager").
3. Egy gyors adatbázis-lekérdezéssel kinyeri az adott pozícióhoz tartozó engedélyeket.
4. Visszaad egy logikai értéket arról, hogy a vizsgált akció engedélyezett-e.

4. Fő Üzleti Folyamatok

A szoftver működése három fő logikai folyamatra bontható, amelyek a teljesítményértékelés teljes életciklusát lefedik a sablonok megtervezésétől kezdve, a dolgozói kitöltésen át a mesterséges intelligenciával támogatott kiértékelésig.

4.1. Kampány- és Űrlapkezelés (Tervezési fázis)

A folyamat a HR munkatársak vagy a vezetők által készített általános űrlapsablonokkal indul. A kérdések összeállítása történhet manuálisan, vagy AI-generálással: utóbbi esetben egy integrált nagy nyelvi modell (LLM) a vállalati alapértékek és az adott pozíció leírása alapján automatikusan releváns, viselkedésalapú kérdéseket hoz létre.

Employees

Campaigns

Form Templates

AI Review

Forms

User Data

Name: Teszt Elek

Position: Vezető

Change password

Logout

Fizikai munkás

Template Questions

Use AI questions

Add New Question

e.g., How satisfied are you?

Text Box

Add Question

Additional details...

Existing Questions

> Question: Felelősségvállalás a munkaterületen

> Question: Mély fókusz a fizikai feladatokban

A sablonok véglegesítése után a vezetők megnyitnak egy Értékelési Kampányt. A rendszer a sablonokból konkrét űrlappéldányokat generál, majd egy hozzárendelési mátrixon keresztül kiosztja azokat a megfelelő dolgozóknak, pontosan meghatározva az értékelői relációkat (ki kit értékkel, pl. vezetői vagy kollégai visszajelzés)

Employees
Campaigns
Form Templates
AI Review
Forms

User Data
Name: Teszt Elek
Position: Vezető

Change password

Logout

Assign Group

Select a group
No group

Select Employees

Teszt Pisti
Teszt Péter
Teszt Vilmos
Teszt Elek
Dói János

Add employee

Everyone has to have self-evaluation
Yes

Random evaluation | How many evaluations per person?
Yes 2

Evaluation Matrix

Form Filler + Target Employee +

Form Filler	Target Employee	Self-evaluation	Back office	Fizikai mu...
Teszt Pisti	Teszt Pisti	☒	☒	☒
Teszt Pisti	Teszt Péter	☒	☒	☒
Teszt Pisti	Teszt Vilmos	☒	☒	☒
Teszt Pisti	Teszt Elek	☒	☒	☒
Teszt Pisti	Dói János	☒	☒	☒
Teszt Péter	Teszt Pisti	☒	☒	☒
Teszt Péter	Teszt Péter	☒	☒	☒
Teszt Péter	Teszt Vilmos	☒	☒	☒
Teszt Péter	Teszt Elek	☒	☒	☒
Teszt Péter	Dói János	☒	☒	☒
Teszt Vilmos	Teszt Pisti	☒	☒	☒
Teszt Vilmos	Teszt Péter	☒	☒	☒
Teszt Vilmos	Teszt Vilmos	☒	☒	☒
Teszt Vilmos	Teszt Elek	☒	☒	☒
Teszt Vilmos	Dói János	☒	☒	☒
Teszt Elek	Teszt Pisti	☒	☒	☒
Teszt Elek	Teszt Péter	☒	☒	☒
Teszt Elek	Teszt Vilmos	☒	☒	☒
Teszt Elek	Teszt Elek	☒	☒	☒
Teszt Elek	Dói János	☒	☒	☒
Dói János	Teszt Pisti	☒	☒	☒
Dói János	Teszt Péter	☒	☒	☒
Dói János	Teszt Vilmos	☒	☒	☒
Dói János	Teszt Elek	☒	☒	☒
Dói János	Dói János	☒	☒	☒

Assign Group

HR System

címzett: én

Hi Teszt,

Please fill out the forms that are assigned to you.

Click the following link to fill out: <http://localhost:8501/?token=b7ab251f-4ab6-42bc-8d3e-ec441f7ff05e>

Thank you!

HR team

4.2. Értékelési és Kitöltési Folyamat (Végrehajtási fázis)

A munkatársak a sikeres bejelentkezést követően egy dedikált felületen látják a még nyitott, rájuk kiosztott értékelési feladatokat. Az űrlap megnyitásakor a Streamlit felület a kérdéstípusoknak megfelelő (szöveges, 0-5 vagy 1-10 skálás) beviteli mezőket jelenít meg. A válaszok leadásakor a rendszer azonnali validációt végez, majd sikeres beküldés esetén tranzakcionálisan letárolja az eredményeket (form_responses és response_answers), a feladat státuszát pedig "be-fejezett"-re (completed) állítja, megakadályozva a további módosításokat.

Employees
Campaigns
Form Templates
AI Review
Forms

User Data
Name: Teszt Elek
Position: Vezető

Change password

Logout

Fizikai munkás

Evaluating: Teszt Vilmos

Fizikai munkás célpontú sablon

Mekkora mértékben törekszik a folyamatok folyamatos javítására?

Ellenőrizze, hogy a dolgozó mennyire kezdi modulárisan a napi feladatokat jobbbá és hatékonyabbá téve.

Röviden írja le, milyen építő jellegű kritikát nyújtott a csapatnak

Közös törekvések előmozdítása - kérjük, egy konkrét példát mutasson be, amikor gondolatot váltottak meg a feladataim tapasztalatai alapján.

Teszt

Press Ctrl+Enter to apply

4.3. AI-alapú Értékelés Generálása (Kiértékelési fázis)

A rendszer legkomplexebb folyamata az emberi visszajelzések automatikus, torzításmentes leképezése. Amikor egy felhasználó lekéri egy dolgozó értékelését, az alkalmazás kigyűjti a célpontra vonatkozó összes leadott választ. Az erőforrás-optimalizáció érdekében a rendszer először képez egy digitális ujjlenyomatot (hash-t) a válaszokból. Ha ez megegyezik egy már korábban legenerált értékeléssel azonnal betölti azt a gyorsítótárból. Új vagy módosult adatok esetén egy LangChain-alapú AI ügynök lép működésbe: a nyers válaszokat egy szigorú rendszer-prompt segítségével elemzi, majd egy strukturált, magyar nyelvű jelentést készít. Ez a Markdown formátumú riport pontozza a kompetenciákat a beérkezett válaszok alapján, meghatározza a fejlesztendő területeket, és összeveti a dolgozó jelenlegi teljesítményét a korábbi kampányok eredményeivel.

The screenshot shows a web application interface for AI review. On the left is a dark sidebar with navigation links: Employees, Campaigns, Form Templates, AI Review (highlighted), and Forms. Below these are 'User Data' fields: Name: Teszt Elek, Position: Vezető, a 'Change password' button, and a red 'Logout' button. The main content area has a purple header with a '← Back to employee list' button and the title 'AI review for Teszt Jóska'. Below the title, the following information is displayed: 'Név: Teszt Jóska', 'Pozíció: Back office (2026-04-09 óta)', 'Munkakör(ök): Lakatos', 'Értékelés dátuma: 2026-06-01-11:13:27', and 'Kampány: Példa Kampány Mindenkinnek 2'. A section titled 'Összesített teljesítményszint' shows an average score of 4.7 on a 0-5 scale. Below this, a section titled 'Teljesítmény összegző értékelése szövegesen' contains a paragraph of feedback text.

5. Felhasználói Felület

A rendszer felhasználói felülete (UI) teljes egészében a Streamlit keretrendszerre épül. A deklaratív, Python-alapú frontend lehetővé teszi az adatvezérelt képernyők gyors renderelését, a reszponzív dizájnt és az üzleti logika zökkenőmentes integrációját anélkül, hogy különálló kliensoldali architektúrára lenne szükség.

5.1. Képernyők és Funkcionális Modulok

A felület funkcionális blokkokra oszlik, amelyek a fő üzleti folyamatokat szolgálják ki. A legfontosabb nézetek:

- **Űrlapsablonok Kezelése:**
 - Listanézet (*show_templates_list*): Megjeleníti az elérhető sablonokat. A nagy adatmennyiség kezelése érdekében beépített lapozással rendelkezik (pl. 10 elem oldalanként).
 - Szerkesztőnézet (*show_edit_template*): Itt adhatók meg az űrlap alapadatainak és kérdéseinek beállításai.
- **Kérdések Szerkesztése:** A komplex kérdéslisták átláthatósága érdekében a rendszer lenyíló paneleket (st.expander) használ. A beállítások strukturált oszlopokban jelennek meg.
- **AI Értékelő Felület:** A *employee_list_view* egy átlátható táblázatban listázza a munkatársakat, ahol az egyes sorokban található legördülő menükből választható ki az adott kampány. A "Get Review" gombra kattintva a rendszer átirányít a *review_view* képernyőre, ahol legenerálja vagy a gyorsítótárból betölti az AI értékelést.

5.2. Állapotvezérelt Navigáció:

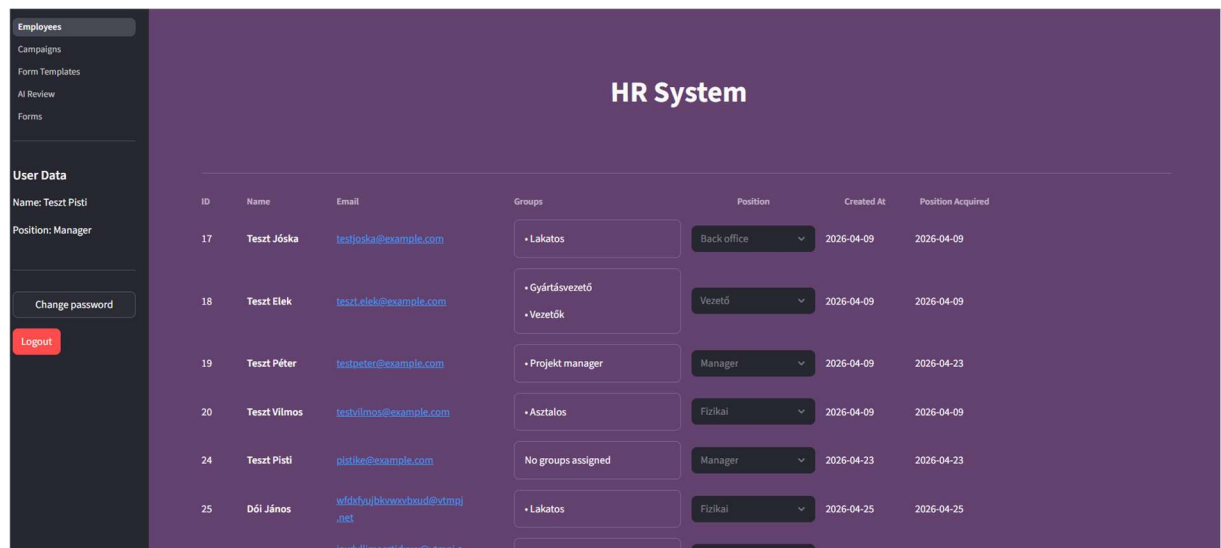
Mivel a Streamlit nem használ hagyományos URL-alapú navigációt, a többképernyős működést a rendszer egy egyedi, állapotvezérelt útválasztó (router) logikával oldja meg a *st.session_state* használatával.

- A navigációt a *set_state()* segédfüggvény irányítja.
- Például az űrlapoknál a *forms_view* kulcs értéke határozza meg a megjelenő képernyőt (pl. *list* vagy *edit_template*).
- Amikor a felhasználó rákattint egy "Edit" gombra, a rendszer beállítja a *current_form_id* értékét a munkamenetben, majd az *st.rerun()* segítségével újratölti a felületet a szerkesztő nézettel. Ugyanez a logika érvényesül az AI értékelőnél a *review_view* és *current_employee_id* változókkal.

5.3. Felhasználói Élmény (UX) és Teljesítményoptimalizálás

A rendszer számos beépített technikát alkalmaz a gyors válaszidők és az interaktív élmény garantálására:

- **Párbeszédpanelek:** A megerősítést igénylő vagy kontextusból kiugró műveleteket (pl. törlés jóváhagyása, AI kérdések generálása, vállalati értékek megadása) a rendszer a modern *@st.dialog* dekorátorral felugró ablakokban kezeli, így a felhasználó nem veszíti el a főképernyő fonalát.
- **Dinamikus Jogosultságkezelés a UI-on:** A biztonsági réteggel összhangban a felületi elemek (gombok, beviteli mezők) vizuálisan is lereagálják a jogosultságokat. Ha a felhasználónak nincs meg a megfelelő joga, a rendszer a *disabled=True* paraméterrel inaktíválja az adott gombot vagy egyáltalán nem látszik a gomb.



The screenshot shows the HR System interface. On the left is a sidebar with navigation links: Employees, Campaigns, Form Templates, AI Review, and Forms. Below these is a 'User Data' section showing 'Name: Teszt Pisti' and 'Position: Manager', with buttons for 'Change password' and 'Logout'. The main area is titled 'HR System' and contains a table of employees. The table has columns for ID, Name, Email, Groups, Position, Created At, and Position Acquired. The data is as follows:

ID	Name	Email	Groups	Position	Created At	Position Acquired
17	Teszt Józsa	testjoka@example.com	• Lakatos	Back office	2026-04-09	2026-04-09
18	Teszt Elek	testelek@example.com	• Gyártásvezető • Vezetők	Vezető	2026-04-09	2026-04-09
19	Teszt Péter	testpeter@example.com	• Projekt manager	Manager	2026-04-09	2026-04-23
20	Teszt Vilmos	testvilmos@example.com	• Asztalos	Fizikai	2026-04-09	2026-04-09
24	Teszt Pisti	pistike@example.com	No groups assigned	Manager	2026-04-23	2026-04-23
25	Dói János	wfdrfuybkwvxbvud@vtmjei.net	• Lakatos	Fizikai	2026-04-25	2026-04-25

• Többszintű Gyorsítótárak:

1. Adatbázis szintű cache: A gyakran ismétlődő, de ritkán változó lekérdezéseket (pl. *get_all_form_templates*, *get_campaign_by_id*) a *@st.cache_data* dekorátor védi időkorláttal.
2. Állapot szintű cache: A sablonok listája a *st.session_state.templates_cache* memóriában is eltárolódik. Új sablon létrehozásakor vagy törlésekor a rendszer ezt a cache-t célzottan érvényteleníti, így minimalizálva az indokolatlan adatbázis-terhelést.

5.4. Vizuális Visszajelzések (Visual Feedback)

A folyamatok nyomon követhetősége érdekében a hosszan futó műveletek (pl. LLM modell meghívása) alatt a UI "töltés" indikátort jelenít meg. A műveletek eredményéről a rendszer színekódolt üzenetekkel tájékoztatja a felhasználót (*st.success* sikeres mentés esetén, *st.warning* hiányzó adatoknál, *st.error* hiba esetén).

6. Hibakezelés és Naplózás

6.1. Üzleti Logika és AI Hibakezelés

A külső rendszerekkel való kommunikáció – kiemelten az LLM API-hívások az OpenRoute-ren keresztül – magában hordozza a hálózati hibák, az időtúllépések vagy a formátumhibák kockázatát.

- **Újrapróbálkozási mechanizmus:** A LangChain integráció beépített `max_retries=5` paraméterrel van konfigurálva, így automatikusan kezeli a túl sok kérést és a pillanatnyi hálózati hibákat.
- **Adatfeldolgozási hibák elkapása:** Mivel az AI válaszok sok esetben strukturált JSON-t (pl. kérdésgenerálás) vagy szigorú Markdown-t várnak el, a kimenet feldolgozása egy try-except blokkban történik. Egy esetleges hiba esetén az alkalmazás nem omlik össze, hanem elkapja a kivételt, és megjeleníti a nyers válaszüzenetet a fejlesztői hibakeresés megkönnyítése érdekében.

6.2. Felhasználói Visszajelzések

A felhasználói felület proaktívan és egyértelműen kommunikál a munkatárssal probléma esetén.

- **Kliensoldali Validáció:** Az űrlapok beküldése vagy módosítása előtt a rendszer ellenőrzi a bemeneteket. Hiányzó kötelező mezők esetén a rendszer nem hívja meg az adatbázist, hanem azonnal *st.warning* vagy *st.error* vizuális üzenettel figyelmezteti a felhasználót.
- **Sikeres műveletek nyugtázása:** A sikeres tranzakciókról és generálásokról a rendszer vizuálisan megerősítő *st.success* üzeneteket küld.
- **Fallbacks:** Ha egy lekért adat nem létezik (pl. rossz ID alapján keresünk űrlapot), az alkalmazás nem dob végzetes hibát, hanem az *st.info* ("No questions added yet") vagy *st.error* ("Employee not found") segítségével biztonságos ágra tereli a folyamatot.

6.3. Naplózás és Telemetry

A rendszer hagyományos naplózását jelenleg egyszerű standard kimenetre írt hibaüzenetek biztosítják (pl. a *db_login.py*-ban). A mesterséges intelligencia működésének megfigyelhetőségére azonban a rendszer eszközöket használ:

- **Phoenix / OpenTelemetry:** Az AI modulok (*AI_form_maker.py*, *AI_review.py*) integrálják az Arize Phoenix lokális tracing szerverét. A `LangChainInstrumentor().instrument()` automatikusan, a háttérben naplózza az összes LLM felé induló kérést, a promptokat, a generálási időket, a felhasznált tokenek számát és a visszaadott válaszokat. A rendszer automatikusan meg is nyitja a lokális böngészőben ezt a trace-felületet.

A megtervezett műszaki alkotás értékelése, továbbfejlesztési lehetőségek

A megtervezett műszaki alkotás sikeresen teljesítette a fő célkitűzéseket: egy modern, jól skálázható és felhasználóbarát platformot hozott létre a vállalati teljesítményértékelési (HR) folyamatok digitalizálására. A rendszer legnagyobb hozzáadott értéke a mesterséges intelligencia funkciók zökkenőmentes integrációja a hagyományos űrlapkezelési folyamatokba és az értékelésbe.

Továbbfejlesztési lehetőségek

Bár a rendszer jelenlegi állapota stabilan kiszolgálja az üzleti folyamatokat, a kódarchitektúra alapján az alábbi technikai és funkcionális fejlesztések valósíthatók meg a jövőre nézve:

- **Adatbázis-kezelés és ORM bevezetése**
Bár a nyers SQL utasítások és az SQLite adatbázis tökéletesen megfelelnek egy kisebb vagy közepes méretű szervezet számára, a rendszer skálázódásával (több ezer munkavállaló, egyidejű konkurens hozzáférések) ez szűk keresztmetszetté válhat. Ezt lehetne orvosolni egy Objektum-Relációs-Leképző (ORM) keretrendszerrel.
- **A rendszer gyorsítása**
A rendszer első indításra (cold start) lassan indul, ezt a sok vizuális elem megjelenítése miatt nem tudja egyből betölteni a streamlit rendszere.
- **Mérhető statisztikák integrálása**
Olyan statisztikák figyelembevétele, amik mérhetőek, hogy még pontosabb legyen az értékelés, például ticketek megoldásának aránya, legyártott eszközök száma.

Irodalomjegyzék

MI ágensek és multiágens rendszerek fejlesztése tárgy ([BME VIK - MI ágensek és multiágens rendszerek fejlesztése](#))

Streamlit dokumentáció (<https://docs.streamlit.io/>)

SMTPlib dokumentáció ([smtplib — SMTP protocol client — Python 3.14.5 documentation](#))

LangChain dokumentáció ([Home - Docs by LangChain](#))

Altair dokumentáció ([Vega-Altair: Declarative Visualization in Python — Vega-Altair 6.1.0dev documentation](#))

Pandas dokumentáció ([pandas - Python Data Analysis Library](#))

OpenRouter oldal ([Models | OpenRouter](#))

SQLite dokumentáció ([SQLite Documentation](#))

Docker manuálok (<https://docs.docker.com/manuals/>)

Általános Python könyvtárak és szintaxis ([3.14.5 Documentation](#))

Google Gemini AI modell (<https://gemini.google.com/>)